

Lập trình Java Spring ORM

GV Nguyễn Đức Kiên



Nội dung bài học

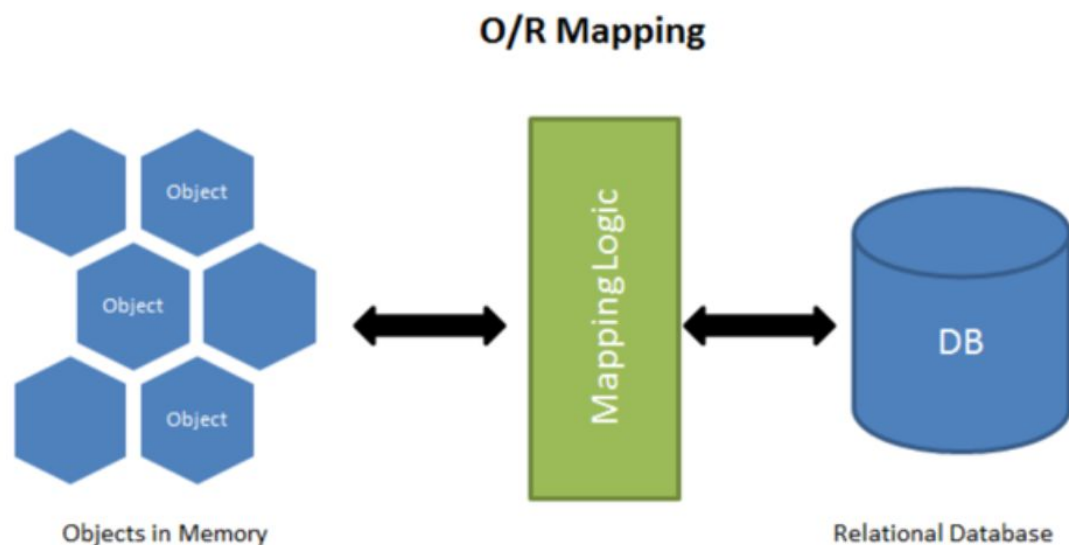
1. Giới thiệu ORM
2. Các Annoation trong ORM
3. Mối quan hệ trong ORM

1) Giới thiệu

1. Định nghĩa
2. Tại sao cần orm, khác biệt khi sử dụng và không sử dụng

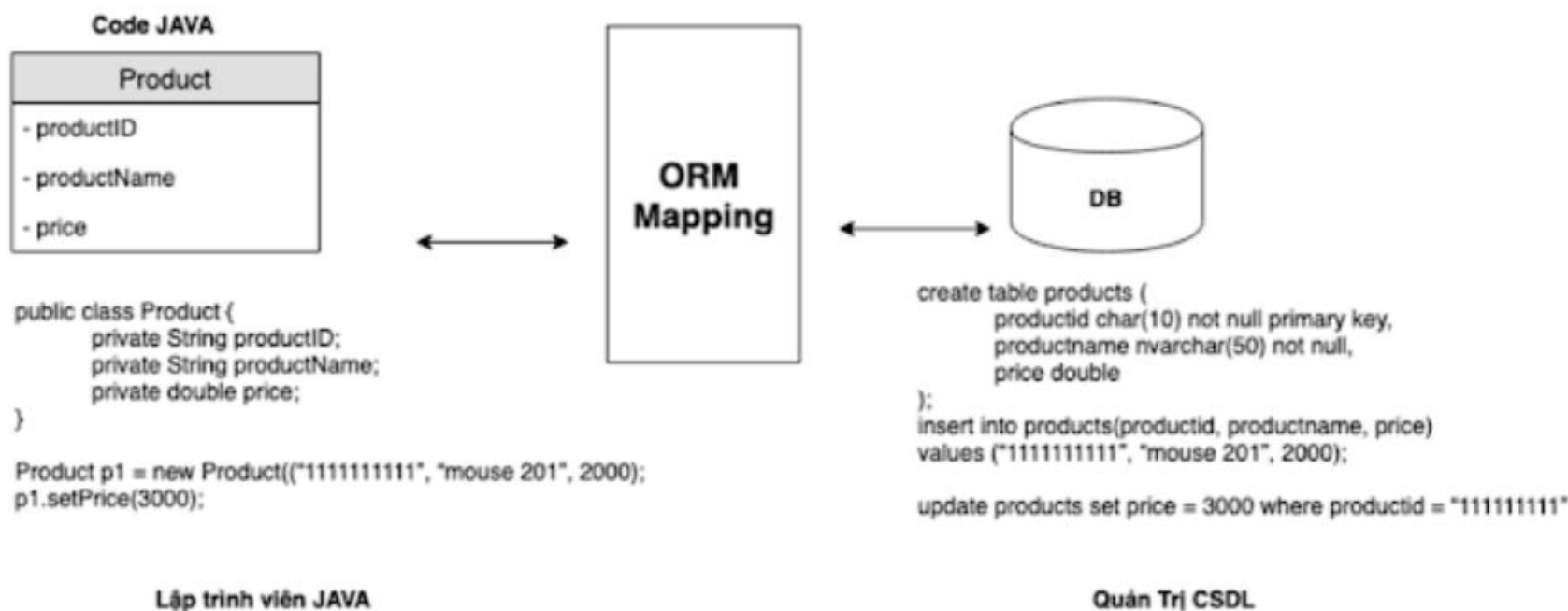
1.1) Định nghĩa

- ORM (Object-Relational Mapping) là một kỹ thuật giúp ánh xạ giữa các đối tượng trong lập trình hướng đối tượng với các bảng trong cơ sở dữ liệu quan hệ.
- Spring ORM cung cấp các công cụ để tích hợp ORM vào các ứng dụng Java, giúp bạn thao tác với cơ sở dữ liệu mà không phải viết quá nhiều mã SQL thủ công.



Ví dụ 1

- Ví dụ: Để lưu trữ danh sách sản phẩm trong CSDL thì người quản trị CSDL và lập trình viên Java sẽ viết theo kiểu của họ sau:



Giải thích về ví dụ 1

- Lớp Product trong Java:
 - ☐ Đây là lớp Java đại diện cho một sản phẩm, với các thuộc tính productID, productName và price.
 - ☐ Trong mã Java, lớp Product có ba biến thành viên (attributes) là productID, productName, và price.
 - ☐ Các thuộc tính này được ánh xạ với các trường trong bảng cơ sở dữ liệu.
 - ☐ Một đối tượng của lớp Product (ở đây là p1) được tạo ra và khởi tạo với các giá trị cụ thể: productID = "1111111111", productName = "mouse 201", và price = 2000.
- Câu lệnh SQL: create table products:
 - ☐ Đây là câu lệnh SQL dùng để tạo bảng products trong cơ sở dữ liệu, với các cột productid, productname, và price.
 - ☐ insert into products: Câu lệnh này thêm một dòng mới vào bảng products với dữ liệu productid = "1111111111", productname = "mouse 201", và price = 2000.
 - ☐ update products: Câu lệnh này cập nhật giá của sản phẩm có productid = "1111111111" lên price = 3000.

Giải thích về ví dụ 1

ORM Mapping:

- ☐ ORM giúp lập trình viên Java làm việc với cơ sở dữ liệu mà không cần phải viết các câu lệnh SQL thủ công.
- ☐ Thay vào đó, ORM sẽ tự động chuyển các đối tượng trong Java thành các dòng trong cơ sở dữ liệu.
- ☐ Mối quan hệ giữa các đối tượng Java và cơ sở dữ liệu được gọi là ORM Mapping. Ở đây, Product trong Java được ánh xạ với bảng products trong cơ sở dữ liệu.
- Tóm tắt:
 - ☐ ORM giúp lập trình viên Java dễ dàng làm việc với cơ sở dữ liệu mà không cần phải viết các câu lệnh SQL trực tiếp ở ví dụ trên chính là giảm thiểu việc viết các câu lệnh sql mapping giữa table products và class Product.
 - ☐ Việc sử dụng ORM giúp giảm thiểu lỗi và tăng tốc quá trình phát triển ứng dụng vì lập trình viên chỉ cần làm việc với các đối tượng Java, trong khi ORM xử lý việc ánh xạ và tương tác với cơ sở dữ liệu.

1.2) Tại sao cần orm, khác biệt khi sử dụng và không sử dụng

- **Vì sao cần ORM:**

- ☐ ORM giúp giảm thiểu mã SQL thủ công, giúp duy trì tính nhất quán và dễ bảo trì mã nguồn.
- ☐ Thay vì phải viết câu lệnh SQL cho từng thao tác với cơ sở dữ liệu, bạn có thể thao tác với các đối tượng Java như bình thường.

- **Khác biệt khi sử dụng và không sử dụng ORM:**

- ☐ Khi không sử dụng ORM: Bạn sẽ phải trực tiếp thao tác với SQL để thêm, sửa, xóa và truy vấn dữ liệu. Điều này dễ gây ra lỗi khi viết mã và khó duy trì khi ứng dụng phát triển.
- ☐ Khi sử dụng ORM: Các thao tác với cơ sở dữ liệu sẽ được quản lý tự động thông qua các đối tượng Java, giúp giảm thiểu các lỗi và tăng khả năng tái sử dụng mã nguồn.

2) Các Annotation trong ORM

Một số annotation hay sử dụng, Các annotation này rất quan trọng trong việc định nghĩa các thực thể và ánh xạ chúng vào cơ sở dữ liệu.

- @Entity
- @Table
- @Id
- @Column
- @JoinColumn
- @GeneratedValue:

2.1) @Entity

- **Định nghĩa:**

- ☐ Annotation @Entity được sử dụng để đánh dấu một lớp Java là thực thể mà ORM sẽ quản lý.
- ☐ Thực thể này sẽ tương ứng với một bảng trong cơ sở dữ liệu. Mỗi đối tượng của lớp này sẽ tương ứng với một dòng trong bảng.

- **Lưu ý:** Lớp Java phải có một constructor không tham số (default constructor). Đây là yêu cầu của JPA (Java Persistence API).
- **Ý nghĩa:** Khi lớp được đánh dấu với @Entity, Hibernate hoặc JPA sẽ quản lý nó như một đối tượng có thể lưu trữ, truy vấn, và xóa từ cơ sở dữ liệu.

```
4  
5  
6  
7  
8  
▼ @Entity 25 usages  kiennnd25  
  @Table(name = "products")  
  public class ProductEntity {
```

2.2) @Table

- **Định nghĩa:** Annotation @Table được sử dụng để chỉ định tên bảng trong cơ sở dữ liệu mà thực thể sẽ ánh xạ vào. Nếu không chỉ định, mặc định bảng sẽ có tên giống với tên lớp.

```
4  
5     @Entity 25 usages  👤 kiennd25  
6     @Table(name = "products")  
7     public class ProductEntity {  
8
```

- **Ý nghĩa:**
- Trong ví dụ trên, @Table(name = "products") chỉ ra rằng thực thể ProductEntity sẽ ánh xạ vào bảng products trong cơ sở dữ liệu.
- Nếu không có annotation @Table, Hibernate sẽ tự động sử dụng tên lớp (trong trường hợp này là ProductEntity) làm tên bảng.

2.3) @Id

- **Định nghĩa:** @Id được sử dụng để chỉ ra rằng trường hoặc thuộc tính được đánh dấu là khóa chính (Primary Key) của bảng trong cơ sở dữ liệu.
- **Cách sử dụng:**

```
5  ✓ @Entity 25 usages  👤 kiennd25
6    @Table(name = "products")
7    public class ProductEntity {
8
9    ✓      @Id
10         @GeneratedValue(strategy = GenerationType.IDENTITY)
11         private int id;
12
```

- **Ý nghĩa:**
 - ☐ @Id đánh dấu rằng trường id là khóa chính của bảng products.
 - ☐ Mỗi thực thể sẽ có một giá trị duy nhất cho trường này.
 - ☐ Trong trường hợp này, giá trị của trường id sẽ được tự động tạo ra và quản lý bởi cơ sở dữ liệu.

2.4) @Column

- **Định nghĩa:** @Column được sử dụng để chỉ định ánh xạ của một thuộc tính Java vào một cột trong bảng cơ sở dữ liệu. Annotation này có thể được sử dụng để tùy chỉnh các thuộc tính của cột như tên cột, độ dài của chuỗi, hay nếu cột có thể nhận giá trị NULL hay không.
- Các sử dụng:

```
5  @Entity 25 usages  kienn25 *
6  @Table(name = "products")
7  public class ProductEntity {
8
9      @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private int id;
12
13     @Column(name = "book_title", nullable = false) 2 usages
14     private String bookTitle;
15
```

2.4) @Column

- **Ý nghĩa:** Trong ví dụ trên, @Column(name = "book_title", nullable = false) ánh xạ thuộc tính bookTitle vào cột book_title trong bảng cơ sở dữ liệu, và cột này không thể có giá trị NULL.
- **Các thuộc tính của @Column:**
 - ☐ name: Tên cột trong cơ sở dữ liệu.
 - ☐ nullable: Chỉ định liệu cột có thể nhận giá trị NULL hay không.
 - ☐ length: Độ dài tối đa của chuỗi cho cột (dùng với kiểu dữ liệu chuỗi như VARCHAR).
 - ☐ unique: Chỉ định liệu giá trị trong cột này có phải là duy nhất hay không.

2.5) @JoinColumn

- **Định nghĩa:**

- ☐ @JoinColumn được sử dụng trong các quan hệ giữa các thực thể, đặc biệt là trong các quan hệ One-to-One và Many-to-One.
- ☐ Nó chỉ định cột trong bảng con mà sẽ dùng để ánh xạ khóa ngoại đến bảng cha.

- **Cách sử dụng**

```
5  @Entity 25 usages  kienn25 *
6  @Table(name = "products")
7  public class ProductEntity {
8
9      @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private int id;
12
13     @Column(name = "book_title", nullable = false) 2 usages
14     private String bookTitle;
15
16     @ManyToOne(fetch = FetchType.LAZY) 2 usages
17     @JoinColumn(name = "category_id")
18     private CategoryEntity category; // Mỗi sản phẩm có một thể loại
19
```

2.5) @JoinColumn

- Ý nghĩa:
 - ☐ Trong ví dụ trên, @JoinColumn(name = "category_id") ánh xạ cột khóa ngoại category_id trong bảng products đến bảng categories.
 - ☐ Cột category_id trong bảng products sẽ lưu trữ khóa ngoại liên kết với bảng categories.
- Các thuộc tính của @JoinColumn:
 - ☐ name: Tên cột khóa ngoại trong bảng con.
 - ☐ referencedColumnName: Tên cột trong bảng cha mà cột khóa ngoại sẽ tham chiếu tới (mặc định là cột khóa chính của bảng cha).
 - ☐ nullable: Xác định liệu cột khóa ngoại có thể là NULL hay không.
 - ☐ unique: Chỉ định liệu cột khóa ngoại có phải là duy nhất hay không.

2.6) @GeneratedValue

- Định nghĩa: @GeneratedValue được sử dụng để chỉ định chiến lược tạo giá trị tự động cho khóa chính. Nó chỉ áp dụng cho các trường đánh dấu với @Id.

```
5  @Entity 25 usages  👤 kiennd25 *  
6  @Table(name = "products")  
7  public class ProductEntity {  
8  
9      @Id  
10     @GeneratedValue(strategy = GenerationType.IDENTITY)  
11     private int id;  
12
```

2.6) @GeneratedValue

- **Ý nghĩa:**

- ☐ @GeneratedValue(strategy = GenerationType.IDENTITY) cho biết rằng giá trị của khóa chính sẽ được tự động tạo ra bởi cơ sở dữ liệu.
- ☐ Ở đây, chiến lược IDENTITY cho phép cơ sở dữ liệu tự động tạo giá trị cho khóa chính (thường được sử dụng trong các cơ sở dữ liệu như MySQL).

- **Các chiến lược trong @GeneratedValue:**

- ☐ GenerationType.IDENTITY: Cơ sở dữ liệu tự động sinh giá trị cho khóa chính.
- ☐ GenerationType.SEQUENCE: Sử dụng một sequence (chuỗi số) trong cơ sở dữ liệu để tạo giá trị cho khóa chính.
- ☐ GenerationType.AUTO: Hibernate tự động chọn chiến lược tốt nhất dựa trên cơ sở dữ liệu đang sử dụng.
- ☐ GenerationType.TABLE: Sử dụng một bảng trong cơ sở dữ liệu để tạo giá trị cho khóa chính.

Tổng kết

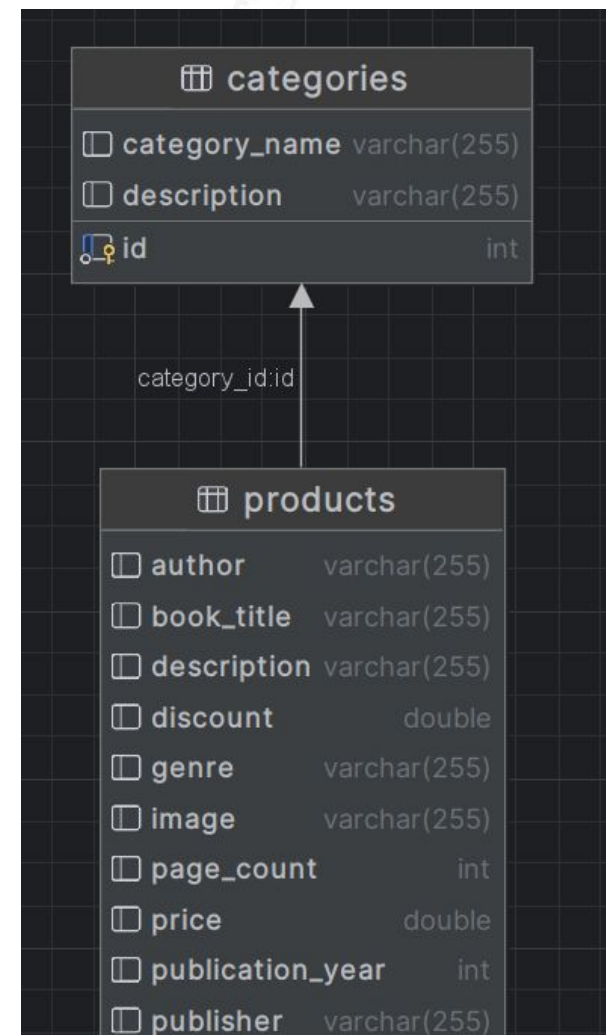
- Các annotation trong ORM giúp chúng ta cấu hình và ánh xạ các đối tượng trong Java với cơ sở dữ liệu một cách dễ dàng và hiệu quả.
- Việc sử dụng đúng các annotation như `@Entity`, `@Table`, `@Id`, `@Column`, `@JoinColumn`, và `@GeneratedValue` giúp việc phát triển ứng dụng Java với cơ sở dữ liệu trở nên rõ ràng, dễ bảo trì và mở rộng.

3) Mối quan hệ trong ORM

- Trong orm hỗ trợ các mối quan hệ sau **Many-to-One**, **One-to-Many**, **Many-to-Many**, và **One-to-One**. Nó là ánh xạ các mối quan hệ giữa các bảng trong cơ sở dữ liệu được đưa lên tầng code Java
- Mỗi mối quan hệ có những đặc điểm và cách triển khai khác nhau

3.1) Many-to-One

- Many-to-One (Nhiều đối tượng này liên kết với một đối tượng khác)
- Định nghĩa: Mỗi quan hệ Many-to-One là khi nhiều đối tượng trong một bảng có thể liên kết với một đối tượng trong bảng khác. Ví dụ, nhiều sản phẩm có thể thuộc về một thể loại.



3.1) Many-to-One

- **Ví dụ trong entity:** Trong trường hợp này, chúng ta có mối quan hệ Many-to-One giữa ProductEntity và CategoryEntity. Một sản phẩm có thể thuộc về một thể loại, nhưng mỗi thể loại có thể chứa nhiều sản phẩm.

```
@Entity 25 usages  👤 kiennd25 *
@Table(name = "products")
public class ProductEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "book_title", nullable = false) 2 usages
    private String bookTitle;

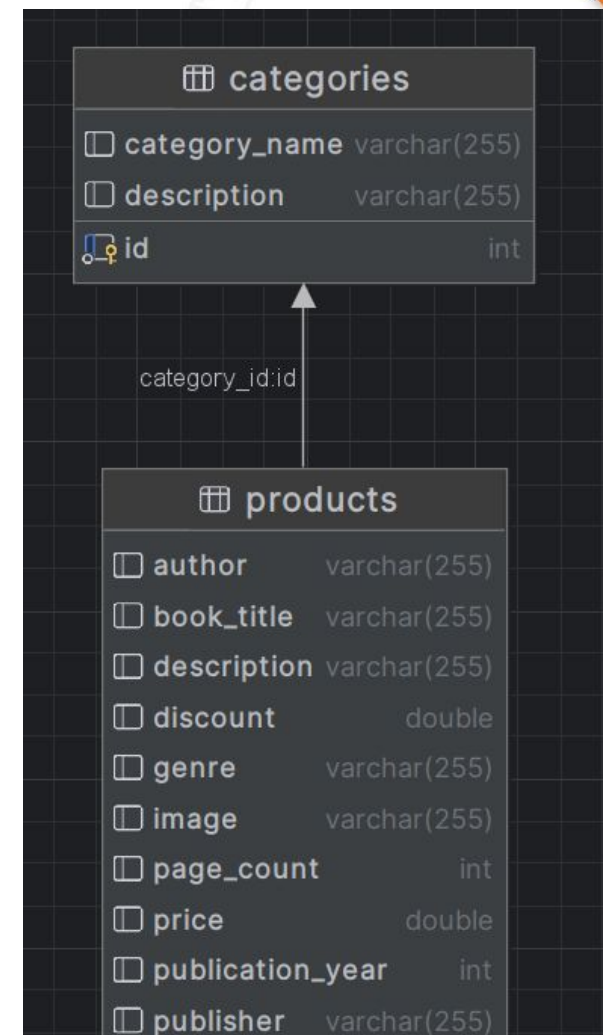
    @ManyToOne(fetch = FetchType.LAZY) 2 usages
    @JoinColumn(name = "category_id")
    private CategoryEntity category; // Mỗi sản phẩm có một thể loại
```

3.1) Many-to-One

- **Giải thích ví dụ:**
- `@ManyToOne` chỉ định rằng mỗi `ProductEntity` sẽ có một `CategoryEntity`.
- Mỗi quan hệ này cũng được ánh xạ với khóa ngoại trong bảng `products` thông qua `@JoinColumn(name = "category_id")`, nơi lưu trữ khóa ngoại tham chiếu đến bảng `categories`.

3.2) One-to-Many

- One-to-Many (Một đối tượng này liên kết với nhiều đối tượng khác)
- **Định nghĩa:** Mỗi quan hệ One-to-Many là khi một đối tượng trong bảng cha có thể liên kết với nhiều đối tượng trong bảng con. Ví dụ, một thể loại có thể có nhiều sản phẩm.



3.2) One-to-Many

- Ví dụ trong entity: Mỗi quan hệ One-to-Many giữa CategoryEntity và ProductEntity. Một thể loại có thể chứa nhiều sản phẩm.

```
7  @Entity 3 usages 1 kienn25 *
8  @Table(name = "categories")
9  public class CategoryEntity {
10
11      @Id
12      @GeneratedValue(strategy = GenerationType.IDENTITY)
13      @Column(name = "id")
14      private int id;
15
16      @Column(name = "category_name") 2 usages
17      private String categoryName;
18
19      @Column(name = "description") 2 usages
20      private String description;
21
22      @OneToMany(mappedBy = "category", fetch = FetchType.LAZY) 2 usages
23      private Set<ProductEntity> products; // danh sách sản phẩm thuộc thể loại này
24
```

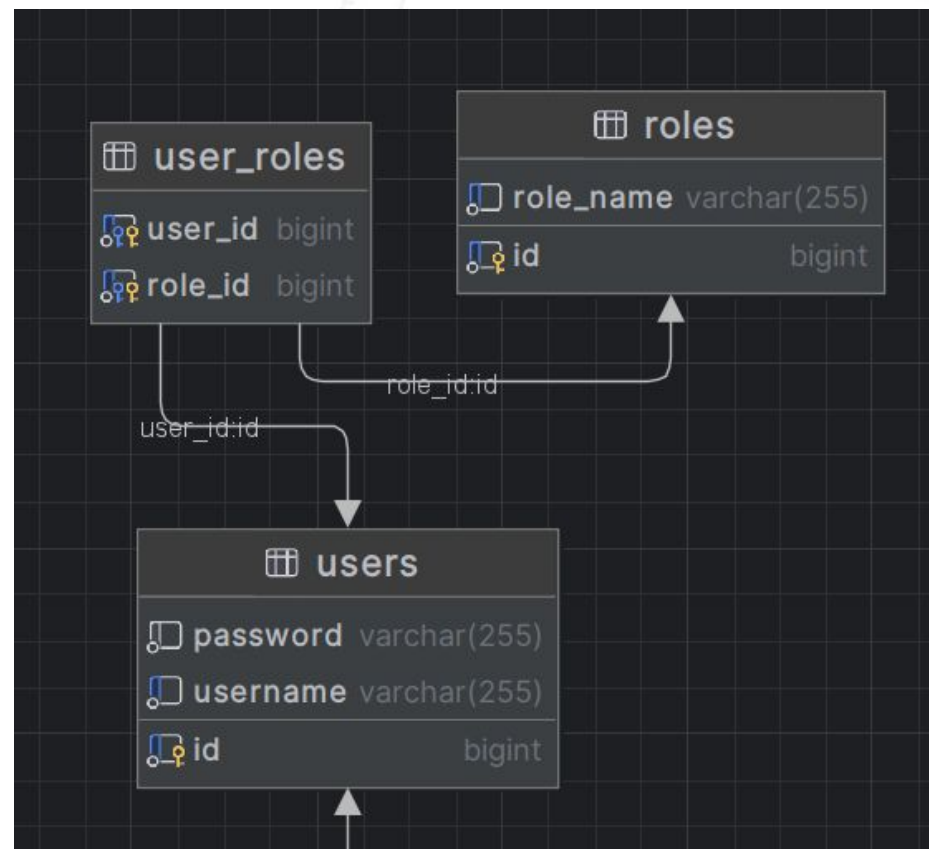
3.2) One-to-Many

Giải thích ví dụ:

- `@OneToMany(mappedBy = "category")` chỉ định rằng mối quan hệ này là One-to-Many và bảng products có trường category để ánh xạ lại về bảng categories.
- `mappedBy = "category"` cho Hibernate biết rằng category trong ProductEntity sẽ chịu trách nhiệm quản lý mối quan hệ này.

3.3) Many-to-Many

- Many-to-Many (Nhiều đối tượng này liên kết với nhiều đối tượng khác)
- **Định nghĩa:** Mỗi quan hệ Many-to-Many là khi nhiều đối tượng trong bảng này có thể liên kết với nhiều đối tượng trong bảng kia.
- **Ví dụ:** một người dùng có thể có nhiều vai trò và một vai trò có thể được gán cho nhiều người dùng.



3.3) Many-to-Many

- Ví dụ trong entity:** Mỗi quan hệ Many-to-Many giữa UserEntity và RoleEntity. Một người dùng có thể có nhiều vai trò, và một vai trò có thể gán cho nhiều người dùng.

```
@Entity 3 usages kiennd25 *
@Table(name = "roles")
public class RoleEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany(mappedBy = "roles") 2 usages
    private Set<UserEntity> users = new HashSet<>();
}
```

```
@Entity 6 usages kiennd25 *
@Table(name = "users")
public class UserEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany 2 usages
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<RoleEntity> roles = new HashSet<>();
}
```

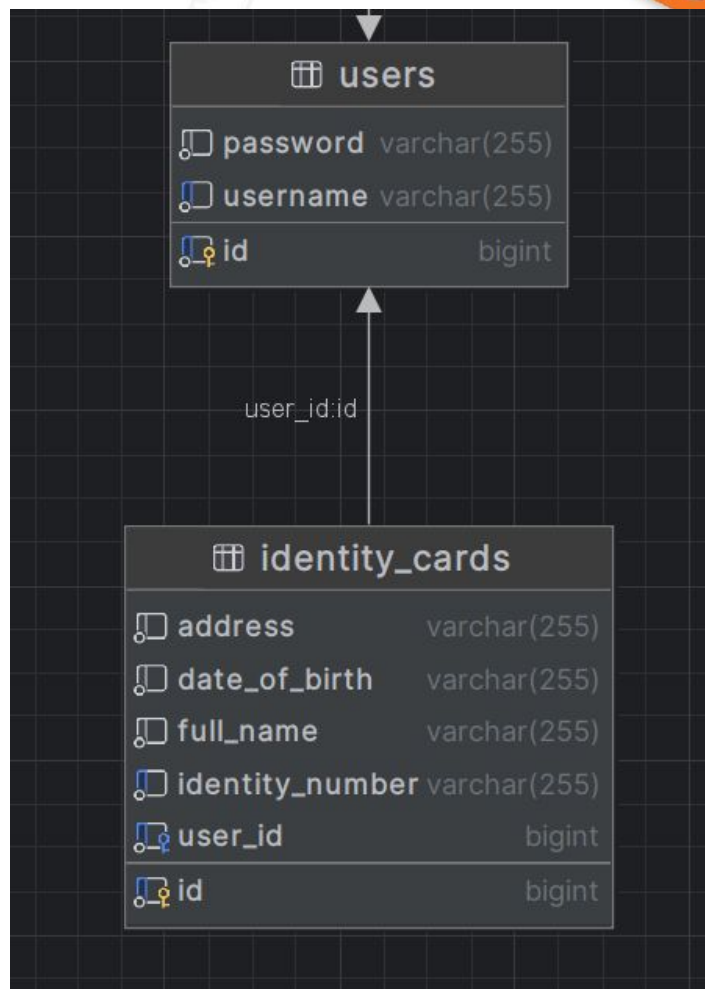

3.3) Many-to-Many

- **Giải thích ví dụ:**

- ❑ @ManyToMany cho biết rằng mối quan hệ này là nhiều - nhiều giữa UserEntity và RoleEntity.
- ❑ @JoinTable chỉ định bảng trung gian user_roles để kết nối bảng users và roles.
- ❑ joinColumns và inverseJoinColumns chỉ ra các cột khóa ngoại trong bảng trung gian.

3.4) One-to-One

- One-to-One (Một đối tượng này chỉ liên kết với một đối tượng khác)
- **Định nghĩa:** Mỗi quan hệ One-to-One là khi mỗi đối tượng trong bảng này chỉ liên kết với một đối tượng trong bảng kia.
- Ví dụ, mỗi người dùng có thể có một thẻ căn cước và ngược lại.



3.4) One-to-One

- **Ví dụ trong entity:** Mỗi quan hệ One-to-One giữa UserEntity và IdentityCardEntity. Một người dùng có thể có một thẻ căn cước.

```
6  @Entity 6 usages 1 kiennd25 *
7  @Table(name = "users")
8  public class UserEntity {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     @OneToOne(mappedBy = "user") 2 usages
15     private IdentityCardEntity identityCard;
16
```

```
1  @Entity 3 usages 1 kiennd25 *
2  @Table(name = "identity_cards")
3  public class IdentityCardEntity {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @OneToOne 2 usages
10     @JoinColumn(name = "user_id", unique = true, nullable = false)
11     private UserEntity user;
12
```

3.4) One-to-One

- **Giải thích:**
- `@OneToOne(mappedBy = "user")` chỉ ra rằng mối quan hệ này là một - một, và trong `IdentityCardEntity`, có một thuộc tính `user` ánh xạ lại đối tượng `UserEntity`.
- Điều này tạo ra một liên kết một - một giữa người dùng và thẻ căn cước.

Tổng kết về các mối quan hệ trong ORM

- **Many-to-One:** Một đối tượng trong bảng cha có thể liên kết với nhiều đối tượng trong bảng con.
- **One-to-Many:** Một đối tượng trong bảng cha có thể liên kết với nhiều đối tượng trong bảng con (được thực hiện qua mappedBy trong bảng con).
- **Many-to-Many:** Nhiều đối tượng trong bảng này có thể liên kết với nhiều đối tượng trong bảng kia, cần một bảng trung gian để lưu trữ các liên kết.
- **One-to-One:** Mỗi đối tượng trong bảng này có thể liên kết với một đối tượng trong bảng kia.

Các mối quan hệ này giúp thiết kế các hệ thống phức tạp với các dữ liệu có liên kết chặt chẽ, tối ưu hóa việc thao tác với cơ sở dữ liệu mà không phải viết nhiều mã SQL thủ công.

4) FetchType

- **FetchType** trong Hibernate (và JPA) điều khiển cách thức tải dữ liệu từ cơ sở dữ liệu. Cụ thể, nó xác định khi nào và làm thế nào các thực thể liên quan (được ánh xạ qua các quan hệ như One-to-Many, Many-to-One, etc.) sẽ được tải.
- **FetchType** giúp tối ưu hóa hiệu suất khi truy vấn dữ liệu từ cơ sở dữ liệu, tránh tải những dữ liệu không cần thiết ngay từ đầu hoặc truy vấn nhiều lần.
- Có 2 loại chính là: EAGER, LAZY

4.1) Fetch Type EAGER

Định nghĩa:

- Khi bạn sử dụng FetchType.EAGER, Hibernate sẽ tải ngay lập tức dữ liệu liên quan khi đối tượng cha được tải.
- Nghĩa là tất cả các mối quan hệ (ví dụ: One-to-Many, Many-to-One) sẽ được nạp ngay khi truy vấn đối tượng chính.

Lưu ý:

- Sử dụng EAGER có thể dẫn đến việc tải quá nhiều dữ liệu không cần thiết, làm giảm hiệu suất nếu bạn chỉ cần một phần của dữ liệu. Có thể khá tốt trong việc giảm bớt số lần tạo connection với database vì dữ liệu đã được lấy lên toàn bộ

4.1) Fetch Type EAGER

Code ví dụ:

- Trong ví dụ trên, khi bạn truy vấn một đối tượng ProductEntity, Hibernate sẽ tự động tải CategoryEntity ngay lập tức (ngay khi truy vấn sản phẩm).

```
5 @Entity 25 usages 1 kienn25 *
6 @Table(name = "products")
7 public class ProductEntity {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private int id;
12
13    @Column(name = "book_title", nullable = false) 2 usages
14    private String bookTitle;
15
16    @ManyToOne(fetch = FetchType.EAGER) 2 usages
17    @JoinColumn(name = "category_id")
18    private CategoryEntity category; // Mỗi sản phẩm có một thể loại
19
```


4.1) Fetch Type LAZY

Định nghĩa:

- Khi sử dụng FetchType.LAZY, Hibernate sẽ không tải các mối quan hệ liên quan ngay lập tức mà chỉ khi thực sự cần thiết (khi bạn truy vấn tới các mối quan hệ đó).

Lưu ý:

- LAZY giúp tiết kiệm bộ nhớ và cải thiện hiệu suất ban đầu, vì chỉ tải dữ liệu khi cần thiết.
- Tuy nhiên, nó có thể tạo ra vấn đề nếu không được quản lý tốt, ví dụ như LazyInitializationException nếu bạn truy cập mối quan hệ sau khi session đã bị đóng.

4.1) Fetch Type LAZY

Code ví dụ:

- Ở đây, mối quan hệ One-to-Many với products sẽ không được tải ngay lập tức khi truy vấn CategoryEntity, mà chỉ khi bạn thực sự truy cập vào thuộc tính products.

```
7  @Entity 3 usages 1 kienn25 *
8  @Table(name = "categories")
9  public class CategoryEntity {
10
11      @Id
12      @GeneratedValue(strategy = GenerationType.IDENTITY)
13      @Column(name = "id")
14      private int id;
15
16      @Column(name = "category_name") 2 usages
17      private String categoryName;
18
19      @Column(name = "description") 2 usages
20      private String description;
21
22      @OneToMany(mappedBy = "category", fetch = FetchType.LAZY) 2 usages
23      private Set<ProductEntity> products; // danh sách sản phẩm thuộc thể loại này
```

Tóm tắt về FetchType:EAGER

- EAGER: Tải dữ liệu liên quan ngay lập tức, có thể gây tải quá mức nếu không sử dụng đúng cách.
- LAZY: Tải dữ liệu khi cần thiết, giúp tối ưu hóa hiệu suất ban đầu nhưng cần phải quản lý tốt để tránh lỗi.

5) Cascade

- Cascade trong Hibernate (và JPA) xác định cách thức các thao tác CRUD (Create, Read, Update, Delete) trên một đối tượng cha sẽ được tự động áp dụng cho các đối tượng con liên quan (đối tượng con trong mỗi quan hệ One-to-Many, Many-to-One, etc.).
- Khi một thao tác (chẳng hạn như persist, merge, remove) được thực hiện trên đối tượng cha, các thao tác đó cũng sẽ được tự động thực hiện trên các đối tượng con, tùy thuộc vào cấu hình cascade.
- Các loại Cascade
 - ☐ CascadeType.PERSIST
 - ☐ CascadeType.MERGE
 - ☐ CascadeType.REMOVE
 - ☐ CascadeType.ALL

5.1) CascadeType.PERSIST

- **Định nghĩa:** Khi persist (thêm mới) được gọi trên đối tượng cha, Hibernate cũng tự động thêm mới các đối tượng con liên quan.
- **Ý nghĩa:** Giúp đảm bảo rằng khi bạn tạo một đối tượng cha, tất cả các đối tượng con liên quan cũng sẽ được lưu vào cơ sở dữ liệu.

5.1) CascadeType.PERSIST

- Có ví dụ:

```
@Entity 3 usages 1 kiennnd25 *
@Table(name = "categories")
public class CategoryEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @OneToMany(mappedBy = "category", cascade = CascadeType.PERSIST, fetch = FetchType.LAZY)
    private Set<ProductEntity> products; // danh sách sản phẩm thuộc thể loại này
```

- Khi persist (thêm mới) một CategoryEntity, tất cả các ProductEntity liên kết với thể loại đó cũng sẽ được persist (lưu) vào cơ sở dữ liệu.

5.2) CascadeType.MERGE:

- **Định nghĩa:** Khi merge (cập nhật) được gọi trên đối tượng cha, Hibernate cũng tự động cập nhật các đối tượng con liên quan.
- **Ý nghĩa:** Giúp đảm bảo rằng khi bạn cập nhật đối tượng cha, các đối tượng con cũng sẽ được cập nhật.

5.2) CascadeType.MERGE:

- Code ví dụ:

```
7  @Entity 3 usages 1 kiennd25 *
8  @Table(name = "categories")
9  public class CategoryEntity {
10
11      @Id
12      @GeneratedValue(strategy = GenerationType.IDENTITY)
13      @Column(name = "id")
14      private int id;
15
16      @OneToMany(mappedBy = "category", cascade = CascadeType.MERGE, fetch = FetchType.LAZY)
17      private Set<ProductEntity> products; // danh sách sản phẩm thuộc thể loại này
18
```

- Khi bạn merge một CategoryEntity, tất cả các ProductEntity liên kết sẽ được cập nhật nếu có thay đổi.

5.3) CascadeType.REMOVE:

- **Định nghĩa:** Khi remove (xóa) được gọi trên đối tượng cha, Hibernate cũng tự động xóa các đối tượng con liên quan.
- **Ý nghĩa:** Giúp đảm bảo rằng khi bạn xóa một đối tượng cha, các đối tượng con cũng sẽ bị xóa, tránh trường hợp dữ liệu không hợp lệ trong cơ sở dữ liệu.

5.3) CascadeType.REMOVE:

- Code demo

```
@Entity 3 usages  🧑  kiennd25 *  
@Table(name = "categories")  
public class CategoryEntity {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    @Column(name = "id")  
    private int id;  
  
    @OneToMany(mappedBy = "category", cascade = CascadeType.REMOVE, fetch = FetchType.LAZY)  
    private Set<ProductEntity> products; // danh sách sản phẩm thuộc thể loại này
```

- Khi remove một CategoryEntity, tất cả các ProductEntity liên kết với thể loại đó cũng sẽ bị xóa khỏi cơ sở dữ liệu.

5.4) CascadeType.ALL

- **Định nghĩa:** Khi ALL được sử dụng, tất cả các thao tác CRUD (persist, merge, remove, refresh, detach) sẽ được tự động áp dụng cho các đối tượng con.
- **Ý nghĩa:** Giúp bạn dễ dàng quản lý các thao tác với cả đối tượng cha và con mà không cần phải chỉ định riêng biệt cho mỗi thao tác.

5.4) CascadeType.ALL

- Code ví dụ:

```
@Entity 3 usages  • kiennd25 *
@Table(name = "categories")
public class CategoryEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private Set<ProductEntity> products; // danh sách sản phẩm thuộc thể loại này
```

- Khi thực hiện bất kỳ thao tác CRUD nào trên CategoryEntity, các thao tác đó cũng sẽ được áp dụng cho tất cả các ProductEntity liên kết.

Tóm tắt về Cascade

- **PERSIST:** Thực hiện thao tác persist trên đối tượng con khi thực hiện persist trên đối tượng cha.
- **MERGE:** Thực hiện thao tác merge trên đối tượng con khi thực hiện merge trên đối tượng cha.
- **REMOVE:** Thực hiện thao tác remove trên đối tượng con khi thực hiện remove trên đối tượng cha.
- **ALL:** Áp dụng tất cả các thao tác CRUD (persist, merge, remove, refresh, detach) cho các đối tượng con khi thực hiện trên đối tượng cha.

